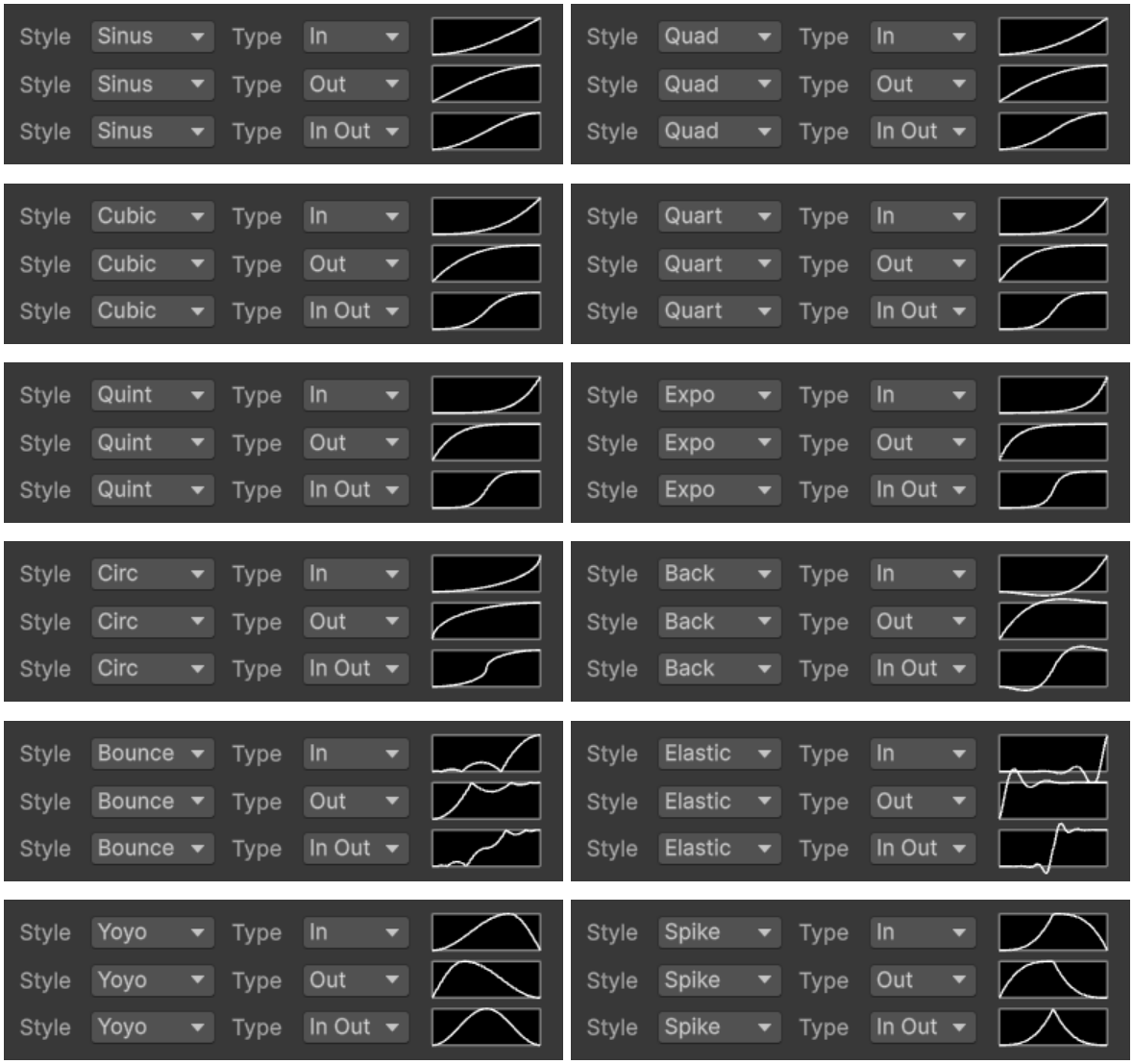
Barliesque.Ease

This small package offers simple, flexible access to all of the standard easing formulas used throughout motion graphics. It allows for several methods of usage, from simply calling a function directly, to selecting ease style as a property in the Inspector. Ease formulas are useful beyond animation, for instance defining a falloff curve.

Ease Styles & Ease Types

The `EaseStyle` enum defines these different styles, each with type In, Out or InOut.



**Yoyo and Spike are experimental additions to the common set of functions*

Using Ease Functions

Combining the easing functions with the `Play()` methods in `GameObjectExtensions.cs` of in the separate package Barliesque.Utils*, you can perform tween animations:

```
private void MoveObject()
{
   this.Play(1f, MoveToEndPos);
}

private void MoveToEndPos(float t)
{
   var eased = Ease.Cubic.InOut(t);
   var pos = Vector3.Lerp(_startPos, _endPos, eased);
   transform.position = pos;
}
```

*See: https://github.com/Barliesque/Utils

In this example, calling `this.Play()` starts a coroutine that executes the callback once per frame, over a period of one second, passing `MoveToEndPos()` a progress value from zero to one. Here, the ease function changes that linear progress value so that it eases in and out, and then the eased progress is used to position the object.

This ease function expects an input value of 0 to 1, and it returns a value of 0 to 1. Since most of Unity's value types like `Vector3, Quaternion, Color,` etc. implement `Lerp()` routines, this approach serves most use cases without having to reimplement the Ease functions for numerous types. However, you can pass in additional start and end values, of the type `float`.

Another way to call the easing functions is `Ease.Call()` which allows you to specify style and type of ease as parameters:

```
var style = EaseStyle.Elastic;
var type = EaseType.Out;

var eased = Ease.Call(style, type, t);
```

Dynamic Ease Selection

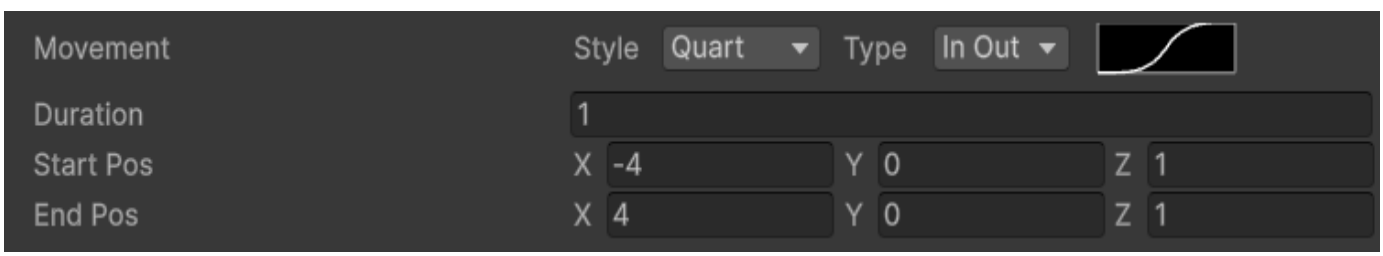
In many cases, it's desirable to not hard code what style of easing is used. With the `EaseSpec` type you can use an Inspector field to select the ease style and type. This is useful if multiple instances of a component want to use different ease settings. It's designer-friendly, allowing ease styles to be easily set without having to write code.

```
[SerializeField] private EaseSpec _movement;
[SerializeField] private float _duration;
[SerializeField] private Vector3 _startPos;
[SerializeField] private Vector3 _endPos;

private void MoveObject()
{
   this.Play(1f, MoveToEndPos);
}

private void MoveToEndPos(float t)
{
   var eased = _movement.Call(t);
   var pos = Vector3.Lerp(_startPos, _endPos, eased);
   transform.position = pos;
}
```

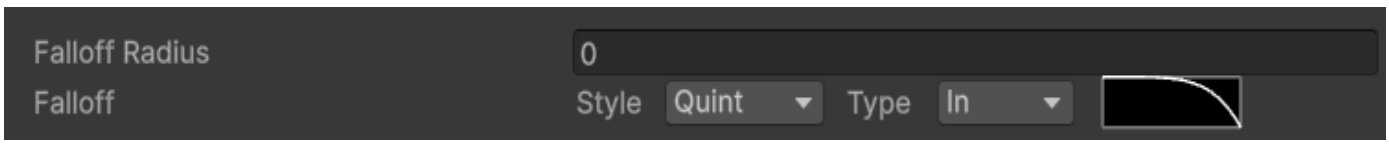
In the inspector, this component appears with a visual representation of the selected ease:



Since `EaseSpec` is a serializable value type, it could be serialized into JSON, if needed.

While, in most cases it makes sense for the preview graph to progress from the lower left to the upper right, some use cases are more appropriately presented going from the upper left to lower right. In the following example, a falloff radius of an explosion would be 100% at the start and 0% at the end. So, using the `[FlipCurve]` meta tag flips the graph in the Inspector, giving a more intuitive visualization.

```
[SerializeField] private float _falloffRadius;
[SerializeField, FlipCurve] private EaseSpec _falloff;
```



Note that this has no effect on how the ease function operates.

More about `this.Play()`

`this.Play()` returns a `CoroutineHandle`. Hang on to that if you want to be able to stop that coroutine before it completes. CoroutineHandle is not part of Unity's Coroutine system, but a 3rd party library called [More Effective Coroutines (MEC)](#)*

*See: https://assetstore.unity.com/packages/tools/animation/more-effective-coroutines-free-54975

Q: *Can I pass in any parameters to the callback function?*

A: Yes! For example:

```
private void MoveObject()
{
   this.Play(1f, MoveToPos, _startPos, _endPos);
}

private void MoveToPos(float t, Vector3 fromPos, Vector3 toPos)
{
   var eased = _movement.Call(t);
   var pos = Vector3.Lerp(fromPos, toPos, eased);
   transform.position = pos;
}
```

Q: *Do I need to extend a specific class that has the Play call?*

A: No. Just add the following using directives at the top of your script:

```
using Barliesque.Easing;
using Barliesque.Utils;
```

Q: *Can it be used inline?*

A: For very simple callbacks, you can write `this.Play` and the callback inline with a delta expression:

```
this.Play(0.5f, (float value) => { /* Your Code Here */ });
```

Q: *How do I stop the animation mid-way?*

A: Using the CoroutineHandle returned from `this.Play()` you can do this:

```
Timing.KillCoroutines(handle);
```

With the Pro version of MEC, you can check `handle.IsRunning` to find out if the coroutine has completed or is still running. You can pause or unpause with: `handle.IsAliveAndPaused = false` or `true`.

For the Pro version of MEC, visit:
https://assetstore.unity.com/packages/tools/animation/more-effective-coroutines-pro-68480

More info about CoroutineHandle, including full documentation, can be found at:
http://trinary.tech/category/mec/